

MUNICIPAL ROBOTICS · RESEARCH

NOVEMBER 2026

Seeing Is Measuring

Editable CAD programs from photographs

CAM PEDERSEN · MUNICIPAL ROBOTICS

Abstract

We recover editable CAD programs (a plate with holes, counterbores, slots, bosses and patterns, each with its dimensions) from photographs by analysis by synthesis: a search over programs scored by description length, with every candidate’s dimensions fitted through a differentiable renderer. Such a fit reports a Fisher error bar for every dimension. We ask whether those error bars, or the fit’s residuals, tell a right answer from a wrong one. On held-out synthetic parts they do not. Among 76 parts, 17 of the 37 whose every error bar is under 1 mm are wrong, and the largest error bar separates wrong answers from right ones with an area under the ROC curve of 0.58. Four image-based checks (residual on the part against the table, outline offset, unexplained residual blobs, score margin to the runner-up) do little better: with thresholds fitted on half of 100 fresh parts, they flag 4 of 37 wrong answers on the other half. Most wrong answers explain the photographs they were fitted to. A photograph the fit has not seen does better, used the right way. Recovering again with one more photograph and flagging the part when its features change catches 30 of 40 wrong answers on 63 fresh parts, with 2 false alarms among 23 right ones, and it works as well on either half of the parts. We have not yet ruled out that some of those changes come from the search’s own run-to-run variation. Scoring the first answer’s residual in that photograph does not (area under the curve 0.51), and choosing which photograph to add, where the finalist programs disagree most, is no better than a random one. We also report a continuous relaxation over features that made identification worse. Every photograph is synthetic.

1 Introduction

A drawing of a part is a short program: “a 69 by 56 by 7 mm plate, chamfered, with a 2.6 mm hole and a boss” is a dozen numbers. Recovering that program from photographs takes two steps. **Identification** decides which features the part has. **Measurement** decides their dimensions. Analysis by synthesis does both by rendering candidate programs and keeping the one whose render best explains the photographs[1,2]. With a differentiable renderer[3] the measurement step is a least-squares fit, and least squares gives every dimension an error bar from the Fisher information.

A user who gets back a program with every dimension at ±0.1 mm will trust it. This paper is about why they should not. The Fisher error bar is the curvature of the score at the answer the search found. It says how well that answer is pinned if the model is right, and nothing about whether the right features were found[4]. A plate 2 mm too short with its hole missed can sit in a basin as narrow as the true one.

We measure how often this happens and whether anything computed from the fit can catch it. Our contributions are:

- **A recovery pipeline** that searches CAD programs by description length[5] through a differentiable renderer, as a testbed (Section 2).
- **A held-out measurement of when its error bars mislead.** On fresh synthetic parts, tight error bars are nearly uninformative about whether the answer is right (Section 4).
- **A negative result for post-hoc checks.** Four residual-based checks, with thresholds fitted on one half of fresh parts and tested on the other, catch few wrong answers. Part 3000 shows why: its wrong answer leaves a clean residual (Section 5).
- **What does help.** More photographs, and a check that uses one: recover again with one more photograph and flag the part if its features change. It catches three in four wrong answers with few false alarms (Section 6, Section 7).

Every photograph here is rendered, so every result excludes what a real camera, print and room would add. The claim is about the fit’s self-assessment, which synthetic data tests cleanly, because the truth is known exactly.

2 The recovery

A program is a plate (width, depth, height, and a sharp, filleted or chamfered corner) with an ordered list of features on its top face (through-holes, counterbores, slots, bosses, linear and circular hole patterns) and its pose on the table. We score a program in nats,

$$\text{score} = \underbrace{-\log p(\text{photos} \mid \text{render}(\text{program}, \theta))}_{\text{residual}} + \underbrace{\frac{1}{2}\log\det F(\theta)}_{\text{parameters}} + \underbrace{-\log p(\text{program})}_{\text{program}},$$

with a Huber noise model, a Laplace parameter cost from the Fisher information $F = J^\top \Sigma^{-1} J$[6,7], and a uniform grammar prior. The noise variance is scaled by the residual’s

autocorrelation area, measured on the open table, so smooth shading errors are not counted as independent pixels.

The search starts from a box, proposes edits (add a feature at a residual blob, remove one, turn a hole into a counterbore or slot, merge holes into a pattern, change the corner), fits each by Levenberg–Marquardt and keeps a beam of two. The best four distinct programs are refitted at full resolution and the lowest score wins. The Jacobian comes from edge sampling[3] in irys[8], through a deterministic photo model (closed-form shading, spherical-harmonic light, one key light with soft shadows, and ambient occlusion whose derivative we take by treating its 16 fixed rays as directional lights). A printed AprilTag[9] fixes each camera and the metric scale.

Test photographs are irys path-traced renders of a part in PLA of one of six colours beside the tag, under a random environment or a synthetic room, at 2016×1512, passed through a phone camera model (defocus, shot and read noise, tone curve, JPEG). Parts are drawn from the grammar's generator. Every setting was frozen on a 20-part development set before any held-out run. Two held-out sets follow: the first held-out set (80 parts, seeds 3020–3099) and the **fresh set** (100 parts, seeds 4000–4099), which we generated after the first set was analysed. The fresh set is reused by the runs of Section 5, Section 6 and Section 7, each a separate run of the search; its Wilson intervals cover the sampling of parts, not the run-to-run variance of the search, which we have not measured.

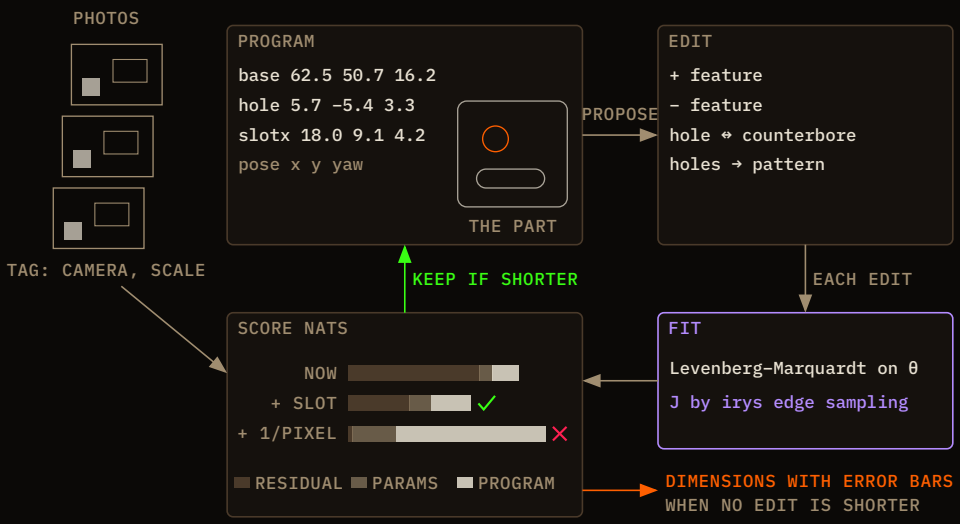


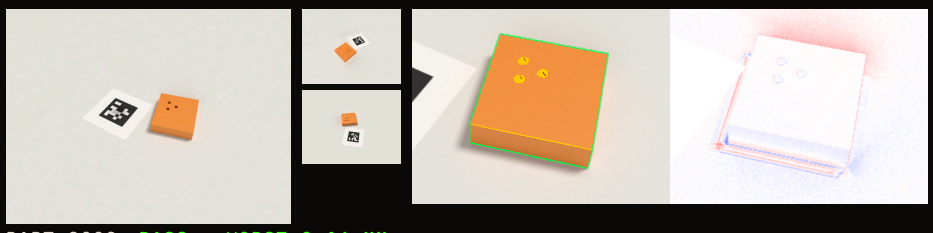
FIG 1 **The recovery loop.** The search edits a CAD program, fits each edit's dimensions through the renderer's edge-sampled Jacobian, scores it in nats, and keeps it only if the description of the photographs gets shorter. Bar lengths are schematic.

3 Accuracy

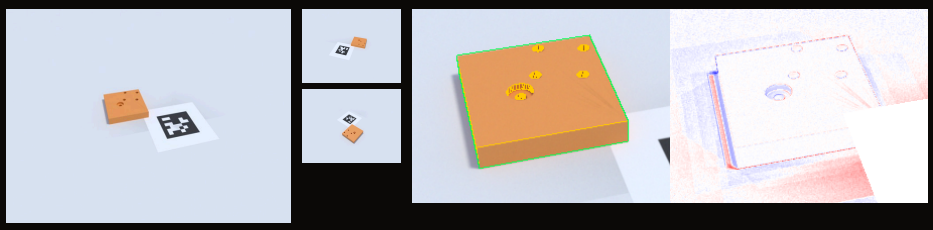
TABLE 1 Recovery accuracy on held-out synthetic parts. Worst dimension per part, after the plate's quarter-turn symmetry. Wilson 95% intervals in parentheses. Most failures are wrong features, chiefly a hole read as a counterbore or the reverse; among parts with the right features the median worst-dimension error is 0.39 mm. `data/measured.tsv`.

SET (3 PHOTOS EACH)	WITHIN 1 MM	RIGHT FEATURES
held-out parts, first run (80)	37 (36–57%)	44 (44–65%)
fresh set, seeds 4000–4099 (100)	41 (32–51%)	54 (44–63%)

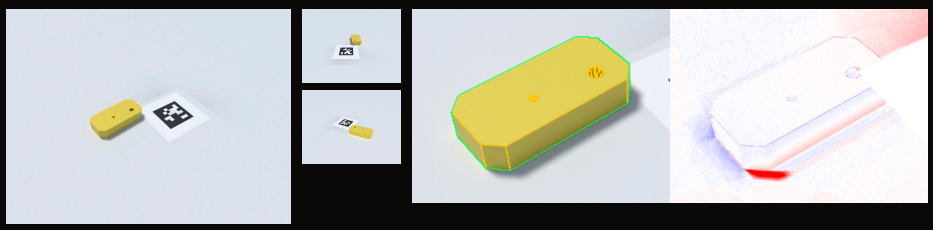
About two in five parts come back within 1 mm on every dimension, and about half with the right features (Table 1). We compare against no other method; the point of this paper is not the accuracy but what the fit says about it. Figure 2 shows two parts it gets right and one it gets wrong.



PART 3009 PASS · WORST 0.16 MM
TRUTH base 74.100 67.500 17.100; circular 3 10.400 10.300 8.600 32.200 3.000
FOUND base 73.998 67.343 16.980; circular 3 10.388 10.196 8.570 32.137 3.013



PART 3012 PASS · WORST 0.32 MM
TRUTH base 65.400 61.100 11.000; circular 4 15.900 -13.000 14.500 30.800 2.500; counterbore -1.500 7.900 2.400 5.600 3.100
FOUND base 65.348 60.992 10.774; circular 4 15.904 -12.950 14.496 120.775 2.494; counterbore -1.450 7.988 2.288 5.584 2.780



PART 3016 FAIL · WRONG FEATURES
TRUTH base 81.900 41.100 16.000 chamfer 7.800; hole -24.500 7.400 3.800; hole 7.600 3.200 2.000
FOUND base 81.490 40.885 15.903 chamfer 7.662; hole 7.560 3.272 2.027; counterbore -24.380 7.465 2.105 3.819 7.940

FIG 2 Three development parts, three synthetic photographs each. Right of the photographs, the fit's overlay: the recovered outline on the crop, and the residual (red brighter, blue darker than the model). Parts 3009 and 3012 pass at 0.16 and 0.32 mm. Part 3016 fails: its hole is read as a counterbore of the hole's outer radius, the commonest confusion. `data/recon-examples.json`.

4 Error bars do not flag wrong answers

The Fisher standard deviations are too small by a median factor of 6 and do not rank the errors (Spearman $\rho = -0.15$). One scale, $\sigma' = 17.8\sigma_F$, fitted on even seeds restores 68% coverage on odd seeds, as any fitted scale must; because σ_F does not rank errors, σ' is an average error with a unit, not a per-dimension error bar. The cause we suspect is bias from the appearance model, which moves an edge the same way in every photograph, so more pixels shrink σ_F without shrinking the error.

The worse failure is that σ is conditional on the features being right. Among the 76 held-out parts that ran (the other 4 of the 80 failed before the search because the tag detector found no tag in a photograph), 37 have $\sigma' \leq 1$ mm on every dimension, and 17 of those are wrong, 14 with the wrong features. As a test for a wrong answer, a part's largest σ has an area under the ROC curve of 0.58 (95% interval 0.45–0.71), where 0.5 is chance. This is the gap between aleatoric and model uncertainty[10], in a setting where the model is a discrete program. Figure 3 shows every part.

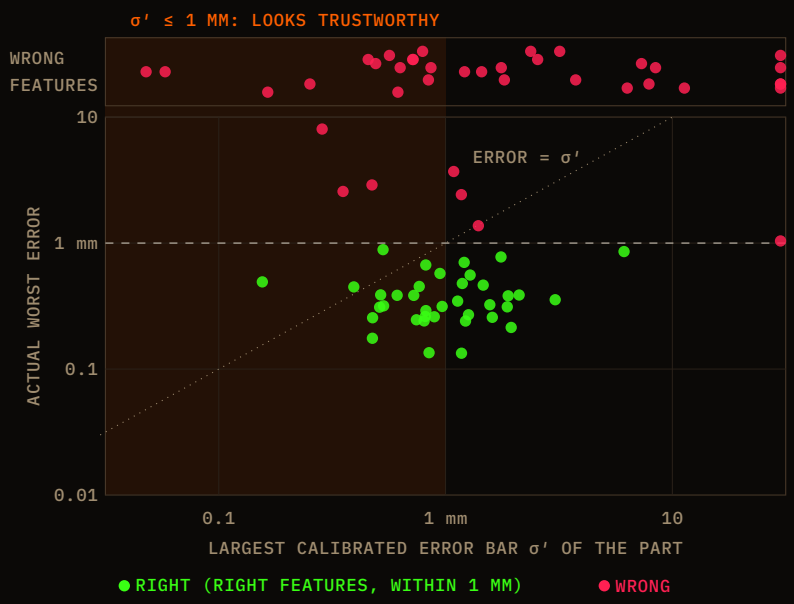


FIG 3 Error bars against errors. Each of the 76 held-out parts at its largest calibrated error bar $\sigma' = 17.8\sigma_F$ and its actual worst error; parts with the wrong features, which have no worst error, sit in the top band. The scale was fitted on the even seeds; the figure shows all parts. In the shaded region every error bar is under 1 mm, and 17 of its 37 parts are wrong. `data/sigma-vs-error.csv`.

5 Residual checks do not either

If the error bars cannot see a wrong program, perhaps the residual can. A missing or misplaced feature should leave residual on the part that the open table does not have. We computed four signals on each finished recovery:

- **residual excess**: mean robust residual near the part over the same on the open table;
- **outline offset**: mean disagreement between the model's silhouette and the segmented part, in pixels;

- **unexplained blob**: the largest residual blob left on the top face, as an equivalent radius;
- **finalist margin**: the score gap in nats from the winner to the runner-up program.

Here a part is wrong if it has the wrong features or any dimension off by more than 1 mm, so the 59 wrong parts of Table 1 include 46 with wrong features. On the fresh set we fitted thresholds on the even seeds, allowing at most 20% false alarms, and tested on the odd seeds. The fitted rule caught 4 of 37 wrong answers with 0 false alarms among 13 right ones. Looser hand-set thresholds caught 14 of 37 at 3 of 13 false alarms. Neither is a usable trust signal (Figure 4). (The odd half happens to be harder, 37 wrong of 50 against 22 on the even half, so these rates are noisy.)

Part 3000 shows why. Its recovered plate is 2.15 mm too narrow and 1.08 mm too tall, and its hole is missing. Every Fisher σ is at most 0.11 mm, the residual on the part is 0.75 times the table's (that is, no excess), and no blob remains. From its photographs' viewpoints the wrong program explains the images. No function of the residual can flag an answer that leaves no residual.



FIG 4 Residual checks on 100 fresh parts. Each dot is a part, split by whether the recovery was right. The two groups overlap on every signal; a threshold that flags many wrong parts flags many right ones too. The fourth signal, an unexplained residual blob, was zero on 97 of the 100 parts and is not drawn; its fitted threshold flags two wrong parts and no right ones. `data/check-fresh.csv`.

6 Photographs do

What separates part 3000's answer from the truth is a view the photographs did not take. On the 20-part development set, one photograph per part put 6 within 1 mm and three photographs 13; the features were right on 11 against 15. On the 100 fresh parts, with the same settings and only the first photograph of each, 16 came back within 1 mm (10–24%) against 41 with three (32–51%), and 27 had the right features against 54. Paired, 27 parts were right with three photographs and wrong with one, and 2 the other way. A third photograph helps accuracy. The next section uses it to say when an answer is wrong.

7 A held-out photograph flags wrong answers

Part 3000 suggests the test: a wrong program that explains two photographs need not explain a third. We tried three ways of using the third photograph, on fresh parts rendered from eight directions each. Every part is recovered from two base views, then again with a third view added. 63 parts finished before we stopped the run; the counts below are on those. They are not the easy ones: in the earlier three-photograph run they were right 24 times in 63 and had the right features 32 times, against 17 and 22 in 37 for the parts that did not finish. Each recovery starts from scratch; the three-photograph recovery is not started from the two-photograph answer.

TABLE 2 **Flagging wrong answers.** A part is flagged when recovering it again with one more photograph changes its features. The last row is the best in-sample check of Section 5, on three-photograph recoveries from a different run of the fresh set and at no extra compute, so it is a reference rather than a like-for-like comparison. `data/measured.tsv`.

CHECK ON THE TWO-PHOTOGRAPH ANSWER	WRONG CAUGHT	RIGHT FLAGGED
features change with a third photograph	**30/40**	2/23
even seeds	13/17	1/15
odd seeds	17/23	1/8
residual checks on the same photographs (Section 5, odd seeds)	4/37	0/13

Does the answer change? Flag the part if the recovery with the third photograph has different features from the recovery without it. This flags (Table 2) 30 of the 40 wrong two-photograph answers (60–86%) and 2 of the 23 right ones. It has no threshold to fit, so we report both halves of the parts: on the even seeds it flags 13 of 17 wrong answers and 1 of 15 right ones, on the odd seeds 17 of 23 and 1 of 8. Among the 31 parts it does not flag, 21 of the two-photograph answers are right, against 23 of 63 overall. It cannot catch an error that the third photograph repeats, which is why a third of the unflagged answers are still wrong.

One confound is not ruled out. The search is not deterministic across machines (Section 9), so some of these changes could be the search landing differently rather than the third photograph adding evidence. Each part's two recoveries ran in the same process on the same machine, but we have not run the control that would settle it: recovering twice from the same two photographs and counting how often the features change. Until then the check is best read as a stability test of the recovery, which is also what a user can run.

The residual in the third photograph. Render the two-photograph answer in the third view, fit that view's light to its own geometry with the geometry held fixed, and measure the residual excess as in Section 5. As a test for a wrong answer it has an area under the curve of 0.51 (95% interval 0.36–0.66): no better than chance. The light absorbs most of what the wrong geometry gets wrong.

Choosing the third photograph. Render the search's finalist programs from each candidate camera and take the view where they disagree most, using only the candidate's camera, not its pixels. With the chosen view 31 parts come back right,

with a fixed random view 32. The chosen view fixed 5 parts the random one missed and missed 6 it fixed; in 14 parts it chose the random view. Adding a view helps; choosing it, this way, does not.

The rule we ship follows: take three photographs, recover from two and from all three, and return the program as trusted only if both have the same features; otherwise ask for another photograph. On these parts it returns about half the parts as trusted, and 21 of those 31 are right (50–81%), against 23 of 63 without the check. It costs one more recovery per part. The residual checks of Section 5 cost nothing extra.

8 A relaxation that made identification worse

Since most failures are a hole-or-counterbore choice, we tried making that choice continuous. Each loose hole or counterbore became a pair of alternatives with amplitudes $a \in [0, 1]$ on a simplex; the model image blends renders without and with each, $I = I_0 + \sum_k a_k (I_k - I_0)$, whose derivative in a_k is exact; the score charges $a_k L_k$ plus an anneal $\beta a(1-a)$ that drives amplitudes to 0 or 1[11,12]. On three small problems it behaved. On the 80 held-out parts, paired against the greedy choice on the same recoveries, it found the right features on 37 against 44, fixing 0 and breaking 7; on the fresh parts, 50 against 54, fixing 1 and breaking 4. The main cause is the anneal: a real hole whose two alternatives split the evidence near 0.45 each loses both. Annealing presence and choice separately ($a_{\text{hole}} = pq$, $a_{\text{cbore}} = p(1-q)$) is the obvious fix and is untested.

9 Limitations

- **Synthetic photographs.** Rendered by the renderer the method is built on. Real photographs add appearance error, which we expect to make error bars less informative, not more.
- **One family of parts**, a plate with top-face features.
- **No baselines.** The accuracy in Table 1 is not compared with any other method.
- **The agreement check's confound.** We have not measured how often the features change when the search runs twice on the same photographs (Section 7), nor tried the held-out residual with the light held from the base views.
- **Single runs.** The greedy search is not deterministic across machines, and we have not measured that variance. The agreement check is measured on the 63 of 100 fresh parts that finished.

10 Conclusion

In analysis-by-synthesis recovery of CAD programs, the fit's own error bars and residuals say little about whether the program is right. On held-out synthetic parts, tight error bars are wrong almost as often as loose ones, and residual checks fitted on one half catch few wrong answers on the other, because most wrong answers explain the photographs they were fitted to. What does help is evidence the fit has not seen, used as a test of stability rather than of fit: recovering again with one

more photograph and asking whether the features change catches three in four wrong answers with few false alarms. A system that reports CAD dimensions with error bars should say plainly that those bars assume the features are right, and should run that check before it does.

The code is the `irys-recon` crate of irys (Apache-2.0); each row of `data/measured.tsv` names the commit and run that produced it.

References

1. Kulkarni, T. D., Kohli, P., Tenenbaum, J. B. & Mansinghka, V. Picture: A Probabilistic Programming Language for Scene Perception. in *IEEE Conference on Computer Vision and Pattern Recognition* (2015).
2. Ellis, K., Ritchie, D., Solar-Lezama, A. & Tenenbaum, J. B. Learning to Infer Graphics Programs from Hand-Drawn Images. in *Advances in Neural Information Processing Systems (NeurIPS)* vol. 31 (2018).
3. Li, T.-M., Aittala, M., Durand, F. & Lehtinen, J. Differentiable Monte Carlo Ray Tracing through Edge Sampling. *ACM Transactions on Graphics* **37,** (2018).
4. White, H. Maximum Likelihood Estimation of Misspecified Models. *Econometrica* **50,** 1–25 (1982).
5. Rissanen, J. Modeling by Shortest Data Description. *Automatica* **14,** 465–471 (1978).
6. Schwarz, G. Estimating the Dimension of a Model. *The Annals of Statistics* **6,** 461–464 (1978).
7. Grünwald, P. D. *The Minimum Description Length Principle*. (MIT Press, 2007).
8. Pedersen, C. Differentiable Rendering with Exact Replay: irys, an Open-Source Spectral Path Tracer. (2026).
9. Olson, E. AprilTag: A Robust and Flexible Visual Fiducial System. in *IEEE International Conference on Robotics and Automation* (2011).
10. Kendall, A. & Gal, Y. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? in *Advances in Neural Information Processing Systems (NeurIPS)* vol. 30 (2017).
11. Liu, H., Simonyan, K. & Yang, Y. DARTS: Differentiable Architecture Search. in *International Conference on Learning Representations* (2019).
12. Louizos, C., Welling, M. & Kingma, D. P. Learning Sparse Neural Networks through L_0 Regularization. in *International Conference on Learning Representations* (2018).